
MODULE *protocol*

Version 2 *Zcash P2P* protocol specification, following the draft *ZIP* “Version 2 *Zcash P2P* Network Protocol” (*zcash/zips* $\neq$ 1344).

Models connection setup and block synchronization over the stream layer of *streams.tla*: the *init* handshake on the dedicated handshake stream, protocol version negotiation, the long-lived block announcement streams (including the rule that at most one announcement stream of a type may be open per direction and the sender’s option to replace a finished or reset stream), and headers-first synchronization over get-headers and get-blocks request streams, each carrying exactly one request and its response.

Blocks are heights on one linear chain: only the peer at the highest height extends it, and a peer that is behind downloads the heights it lacks.

Communication keeps the legacy model’s message consumption discipline: a peer decides only from its own records; the only place both peers’ records are written together is the transport itself (connection open/close, stream open, and the freeing of a stream slot once both sides have closed it).

Two boolean constants select how a receiver interprets the draft:

<i>StrictSingleton</i>	TRUE	a second open announcement stream of the same type from a peer is a <i>PROTOCOL_ERROR</i> (literal reading of “Announcement Streams”)
	FALSE	the receiver accepts it and drains both
<i>RefusePreHandshake</i>	TRUE	announcement streams that arrive before the local handshake completes are refused with <i>REFUSED</i> (“Connection Handshake”: MAY refuse)
	FALSE	they are buffered until it completes

Under *StrictSingleton* the invariant *NoHonestProtocolError* is violated: two conformant peers disconnect because stream independence lets a replacement stream’s type byte arrive before the old stream’s *FIN* or reset.

See *documents/v2-modeling.md* for the full write-up.

EXTENDS *TLC*, *Naturals*, *Sequences*, *FiniteSets*, *streams*, *records*

CONSTANT <i>InitialPeers</i>	set of peers
CONSTANT <i>MaxBlock</i>	maximum block height (initial and mined)
CONSTANT <i>MaxRestarts</i>	finish/reset of own announcement streams, per peer pair
CONSTANT <i>MaxHeaders</i>	headers per get-headers response (draft: 160)
CONSTANT <i>MaxBlocksPerRequest</i>	hashes per get-blocks request (draft: 128)
CONSTANT <i>MinVersion</i>	minimum protocol version of the draft (not yet assigned)
CONSTANT <i>Versions</i>	protocol versions a peer may advertise
CONSTANT <i>StrictSingleton</i>	see module comment
CONSTANT <i>RefusePreHandshake</i>	see module comment

VARIABLE *nodes*

vars $\triangleq$ $\langle nodes \rangle$

See *README* for an explanation of symmetry reduction.
Symmetry $\triangleq$ *Permutations*(*InitialPeers*)

For each initial peer construct a set of all other peers.

$$OtherPeers \triangleq [n \in InitialPeers \mapsto InitialPeers \setminus \{n\}]$$

$$Min(a, b) \triangleq \text{IF } a < b \text{ THEN } a \text{ ELSE } b$$

$$Max(a, b) \triangleq \text{IF } a > b \text{ THEN } a \text{ ELSE } b$$

$$Init \triangleq$$

$$\exists blockset \in [InitialPeers \rightarrow (1 \dots MaxBlock)] :$$

$$\exists verset \in [InitialPeers \rightarrow Versions] :$$

$$\begin{aligned} nodes = [i \in InitialPeers \mapsto [\\ & conn \quad \mapsto [j \in OtherPeers[i] \mapsto \text{"none"}], \\ & close \quad \mapsto [j \in OtherPeers[i] \mapsto NoCode], \\ & streams \mapsto [j \in OtherPeers[i] \mapsto [sid \in StreamIds(i, j) \mapsto NullStream]], \\ & init_sent \mapsto [j \in OtherPeers[i] \mapsto FALSE], \\ & init_recvd \mapsto [j \in OtherPeers[i] \mapsto FALSE], \\ & version \mapsto [j \in OtherPeers[i] \mapsto 0], \\ & peer_tip \mapsto [j \in OtherPeers[i] \mapsto 0], \\ & announced \mapsto [j \in OtherPeers[i] \mapsto 0], \\ & restarts \mapsto [j \in OtherPeers[i] \mapsto 0], \\ & want \quad \mapsto [j \in OtherPeers[i] \mapsto \langle \rangle], \\ & score \quad \mapsto [j \in OtherPeers[i] \mapsto 0], \\ & blocks \quad \mapsto 1 \dots blockset[i], \\ & my_version \mapsto verset[i] \\]] \end{aligned}$$

— Helpers —

$$Height(n) \triangleq Cardinality(nodes[n].blocks)$$

$$Connected(n, m) \triangleq nodes[n].conn[m] \in \{\text{"initiator"}, \text{"responder"}\}$$

The handshake is complete for n once it has both sent and received an *init* record (draft: “Handshake Sequence”, step 3).

$$HandshakeComplete(n, m) \triangleq nodes[n].init_sent[m] \wedge nodes[n].init_recvd[m]$$

$$S(n, m, sid) \triangleq nodes[n].streams[m][sid]$$

Slots n may use to open a new stream toward m (lowest free slot is taken).

$$FreeSlots(n, m) \triangleq \{k \in 1 \dots MaxStreams : S(n, m, \langle n, k \rangle).status = \text{"none"}\}$$

$$FreeSlot(n, m) \triangleq \text{CHOOSE } k \in FreeSlots(n, m) : \forall j \in FreeSlots(n, m) : k \leq j$$

Announcement streams of type t that n currently sends to m on.

$$\begin{aligned} LiveAnn(n, m, t) \triangleq \{sid \in StreamIds(n, m) : \\ & \wedge sid[1] = n \\ & \wedge S(n, m, sid).status = \text{"open"} \\ & \wedge S(n, m, sid).rtype = t \end{aligned}$$

$$\wedge S(n, m, sid).out = \text{"open"}\}$$

Request streams n has outstanding toward m .

$$\begin{aligned} OpenReq(n, m) &\triangleq \{sid \in StreamIds(n, m) : \\ &\quad \wedge sid[1] = n \\ &\quad \wedge S(n, m, sid).status = \text{"open"} \\ &\quad \wedge S(n, m, sid).rtype \in RequestTypes\} \end{aligned}$$

The sequence of heights $a \dots b$ (empty when $b < a$).

$$Range(a, b) \triangleq [i \in 1 \dots (b - a + 1) \mapsto a + i - 1]$$

$$Contiguous(seq) \triangleq \forall i \in 1 \dots (Len(seq) - 1) : seq[i + 1] = seq[i] + 1$$

Heights of seq above h .

$$Above(seq, h) \triangleq SelectSeq(seq, LAMBDA x : x > h)$$

Handshake streams n knows about on its connection with m .

$$\begin{aligned} HandshakeStreams(n, m) &\triangleq \{sid \in StreamIds(n, m) : \\ &\quad \wedge S(n, m, sid).status = \text{"open"} \\ &\quad \wedge S(n, m, sid).rtype = HandshakeType\} \end{aligned}$$

Transport: once both peers have closed a stream its slot is freed.

$$\begin{aligned} Settle(ns, n, m, sid) &\triangleq \\ \text{IF } &\wedge ns[n].streams[m][sid].status = \text{"closed"} \\ &\wedge ns[m].streams[n][sid].status = \text{"closed"} \\ \text{THEN } &[ns \text{ EXCEPT } ![n].streams[m][sid] = NullStream, \\ &\quad ![m].streams[n][sid] = NullStream] \\ \text{ELSE } &ns \end{aligned}$$

Transport: n closes the connection to m with an application error code.

Both peers observe the close; data in flight is lost.

$$\begin{aligned} Close(ns, n, m, code) &\triangleq \\ [ns \text{ EXCEPT } &![n].conn[m] = \text{"closed"}, \\ &![n].close[m] = code, \\ &![n].streams[m] = [sid \in StreamIds(n, m) \mapsto NullStream], \\ &![m].conn[n] = \text{"closed"}, \\ &![m].close[n] = code, \\ &![m].streams[n] = [sid \in StreamIds(n, m) \mapsto NullStream]] \end{aligned}$$

— Connection —

n dials m . The *QUIC* handshake is out of scope and establishment is atomic:

n becomes the initiator and m the responder of the application handshake.

$$\begin{aligned} Connect &\triangleq \\ \exists n \in &InitialPeers : \\ \exists m \in &OtherPeers[n] : \end{aligned}$$

$$\begin{aligned}
& \wedge nodes[n].conn[m] = \text{"none"} \\
& \wedge nodes[m].conn[n] = \text{"none"} \\
& \wedge nodes' = [nodes \text{ EXCEPT } ![n].conn[m] = \text{"initiator"}, \\
& \hspace{15em} ![m].conn[n] = \text{"responder"}]
\end{aligned}$$

— Handshake —

The initiator opens the handshake stream and sends its *init* record on it (draft: “Handshake Sequence”, step 1). The type byte and the record are queued together: they travel in order on the same stream.

$$\begin{aligned}
OpenHandshakeStream & \triangleq \\
& \exists n \in InitialPeers : \\
& \quad \exists m \in OtherPeers[n] : \\
& \quad \quad \wedge nodes[n].conn[m] = \text{"initiator"} \\
& \quad \quad \wedge \neg nodes[n].init_sent[m] \\
& \quad \quad \wedge FreeSlots(n, m) \neq \{\} \\
& \quad \quad \wedge LET \ sid \triangleq \langle n, FreeSlot(n, m) \rangle \\
& \quad \quad IN \quad nodes' = [nodes \text{ EXCEPT } \\
& \quad \quad \quad ![n].streams[m][sid] = OpenerStream(HandshakeType), \\
& \quad \quad \quad ![m].streams[n][sid] = PeerStream(HandshakeType, \\
& \quad \quad \quad \quad \langle MakeInit(nodes[n].my_version, Height(n)) \rangle), \\
& \quad \quad \quad ![n].init_sent[m] = TRUE]
\end{aligned}$$

The responder sends its own *init* record on the handshake stream. It may do so as soon as it has seen the stream’s type byte, without waiting for the initiator’s *init* record (draft: “Handshake Sequence”).

$$\begin{aligned}
SendInitResponder & \triangleq \\
& \exists n \in InitialPeers : \\
& \quad \exists m \in OtherPeers[n] : \\
& \quad \quad \exists sid \in HandshakeStreams(n, m) : \\
& \quad \quad \quad \wedge nodes[n].conn[m] = \text{"responder"} \\
& \quad \quad \quad \wedge \neg nodes[n].init_sent[m] \\
& \quad \quad \quad \wedge nodes' = [nodes \text{ EXCEPT } \\
& \quad \quad \quad \quad ![m].streams[n][sid] = PutData(@, MakeInit(nodes[n].my_version, Height(n))), \\
& \quad \quad \quad \quad ![n].init_sent[m] = TRUE]
\end{aligned}$$

n consumes the type byte of a stream *m* opened. What happens depends on the type and on *n*’s own state:

0x00 from the responder, or a second 0x00 $\rightarrow$ *PROTOCOL_ERROR* (“Connection Handshake”)
any other stream before *n*’s handshake completed $\rightarrow$ refuse with *REFUSED*, or wait (*RefusePreHandshake*)
announcement duplicating an open one of type *t* $\rightarrow$ *PROTOCOL_ERROR*, or accept (*StrictSingleton*)
otherwise $\rightarrow$ record the type

$$\begin{aligned}
RecvTypeByte & \triangleq \\
& \exists n \in InitialPeers :
\end{aligned}$$

announcement (draft: “Block Announcements”, header announcement).

$SendAnnouncement \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in LiveAnn(n, m, BlockAnnType) :$

$\wedge nodes[n].announced[m] < Height(n)$

$\wedge nodes' = [nodes \text{ EXCEPT}$

$! [m].streams[n][sid] = PutData(@, MakeHeaderAnnouncement(Height(n))),$

$! [n].announced[m] = Height(n)]$

n consumes a header announcement from m . Announcements are only processed once n 's handshake is complete (draft: “Connection Handshake”).

$RecvAnnouncement \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in StreamIds(n, m) :$

$LET s \triangleq S(n, m, sid)$

IN

$\wedge Connected(n, m)$

$\wedge HandshakeComplete(n, m)$

$\wedge s.status = \text{“open”}$

$\wedge s.rtype \in AnnTypes$

$\wedge Len(s.inq) > 0$

$\wedge Head(s.inq).kind = \text{“header”}$

$\wedge nodes' = [nodes \text{ EXCEPT}$

$! [n].streams[m][sid].inq = Tail(@),$

$! [n].peer_tip[m] = Max(@, Head(s.inq).height)]$

n finishes one of its announcement streams. The draft allows a sender to open a replacement afterwards; why it finished (backpressure, internal restart) is not modeled. Bounded by $MaxRestarts$ to keep liveness meaningful.

$FinishAnnouncementStream \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in LiveAnn(n, m, BlockAnnType) :$

$\wedge nodes[n].restarts[m] < MaxRestarts$

$\wedge nodes' = Settle([nodes \text{ EXCEPT}$

$! [n].streams[m][sid] = ClosedStream,$

$! [m].streams[n][sid] = PutData(@, FIN),$

$! [n].restarts[m] = @ + 1],$

$n, m, sid)$

Same as $FinishAnnouncementStream$ but via $RESET_STREAM$, which can overtake records still in flight on that stream.

$ResetAnnouncementStream \triangleq$

$\exists n \in InitialPeers :$

$$\begin{aligned}
& \exists m \in \text{OtherPeers}[n] : \\
& \quad \exists \text{sid} \in \text{LiveAnn}(n, m, \text{BlockAnnType}) : \\
& \quad \quad \wedge \text{nodes}[n].\text{restarts}[m] < \text{MaxRestarts} \\
& \quad \quad \wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PutReset}(@, \text{"INTERNAL_ERROR"}), \\
& \quad \quad \quad ! [n].\text{restarts}[m] = @ + 1], \\
& \quad \quad n, m, \text{sid})
\end{aligned}$$

— Request streams: headers-first synchronization —

n is behind m (from m 's *init* or announcements), has nothing queued to download and no request outstanding: it asks m for headers after its own tip (draft: "Headers-First Synchronization", step 1). The request and the *FIN* of n 's sending direction are written together.

$\text{SendGetHeaders} \triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \wedge \text{Connected}(n, m) \\
& \quad \quad \wedge \text{HandshakeComplete}(n, m) \\
& \quad \quad \wedge \text{nodes}[n].\text{peer_tip}[m] > \text{Height}(n) \\
& \quad \quad \wedge \text{nodes}[n].\text{want}[m] = \langle \rangle \\
& \quad \quad \wedge \text{OpenReq}(n, m) = \{ \} \\
& \quad \quad \wedge \text{FreeSlots}(n, m) \neq \{ \} \\
& \quad \quad \wedge \text{LET } \text{sid} \triangleq \langle n, \text{FreeSlot}(n, m) \rangle \\
& \quad \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{RequesterStream}(\text{GetHeadersType}), \\
& \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PeerStream}(\text{GetHeadersType}, \\
& \quad \quad \quad \langle \text{MakeGetHeaders}(\text{Height}(n)), \text{FIN} \rangle)]
\end{aligned}$$

n serves a get-headers request from m : the headers after the locator, at most *MaxHeaders* of them, then *FIN*. Serving may start as soon as the request is complete, before m 's *FIN* has been consumed (draft: "Request Streams").

$\text{ServeGetHeaders} \triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists \text{sid} \in \text{StreamIds}(n, m) : \\
& \quad \quad \quad \text{LET } s \triangleq S(n, m, \text{sid}) \\
& \quad \quad \text{IN} \\
& \quad \quad \quad \wedge \text{Connected}(n, m) \\
& \quad \quad \quad \wedge \text{HandshakeComplete}(n, m) \\
& \quad \quad \quad \wedge s.\text{status} = \text{"open"} \\
& \quad \quad \quad \wedge s.\text{rtype} = \text{GetHeadersType}
\end{aligned}$$

n consumes a headers response from m (draft: “get-headers”):

more than $MaxHeaders$, or not contiguous	→ discard, misbehavior penalty
otherwise	→ queue the heights n lacks

n requests the next batch of blocks it wants from m , at most $MaxBlocksPerRequest$ of them (draft: “get-blocks”).

9

$Min(MaxBlocksPerRequest, Len(nodes[n].want[m]))$

IN $nodes' = [nodes \text{ EXCEPT}$
 $! [n].streams[m][sid] = RequesterStream(GetBlocksType),$
 $! [m].streams[n][sid] = PeerStream(GetBlocksType,$
 $\langle MakeGetBlocks(batch), FIN \rangle)]$

n serves a get-blocks request from m . It may finish after any complete entry, delivering fewer blocks than requested (draft: “get-blocks”); the requester re-requests the remainder.

$ServeGetBlocks \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in StreamIds(n, m) :$

LET $s \triangleq S(n, m, sid)$

IN

$\wedge Connected(n, m)$

$\wedge HandshakeComplete(n, m)$

$\wedge s.status = \text{“open”}$

$\wedge s.rtype = GetBlocksType$

$\wedge s.out = \text{“open”}$

$\wedge Len(s.inq) > 0$

$\wedge Head(s.inq).kind = \text{“get-blocks”}$

$\wedge \text{LET } asked \triangleq Head(s.inq).heights$

$held \triangleq SelectSeq(asked, \text{LAMBDA } h : h \in nodes[n].blocks)$

IN

$\exists k \in 1 \dots Len(held) :$

$nodes' = [nodes \text{ EXCEPT}$

$! [n].streams[m][sid] = Collapse([@ \text{ EXCEPT } !.inq = Tail(@),$
 $!.out = \text{“finished”}]),$

$! [m].streams[n][sid] = PutData(PutData(@, MakeBlocks(SubSeq(held, 1, k))), FIN)]$

n consumes delivered blocks from m and extends its chain with them.

$RecvBlocks \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in OpenReq(n, m) :$

LET $s \triangleq S(n, m, sid)$

IN

$\wedge Connected(n, m)$

$\wedge s.rtype = GetBlocksType$

$\wedge Len(s.inq) > 0$

$\wedge Head(s.inq).kind = \text{“blocks”}$

$\wedge \text{LET } got \triangleq Head(s.inq).heights$

$blocks \triangleq nodes[n].blocks \cup \{got[i] : i \in 1 \dots Len(got)\}$

IN $nodes' = [nodes \text{ EXCEPT}$

$$\begin{aligned} & \vee \wedge s.rtype \neq HandshakeType \\ & \wedge nodes' = Settle([nodes \text{ EXCEPT } ![n].streams[m][sid] = ClosedStream], \\ & \quad n, m, sid) \end{aligned}$$

n observes a *STOP_SENDING* from m on a stream n is sending on, and resets its sending direction as the draft recommends (“Request Streams”).

$$\begin{aligned} RecvStop & \triangleq \\ & \exists n \in InitialPeers : \\ & \quad \exists m \in OtherPeers[n] : \\ & \quad \quad \exists sid \in StreamIds(n, m) : \\ & \quad \quad \quad LET \ s \triangleq S(n, m, sid) \\ & \quad \quad \quad IN \\ & \quad \quad \quad \wedge Connected(n, m) \\ & \quad \quad \quad \wedge sid[1] = n \\ & \quad \quad \quad \wedge s.status = \text{“open”} \\ & \quad \quad \quad \wedge s.out = \text{“open”} \\ & \quad \quad \quad \wedge s.stop \neq NoCode \\ & \quad \quad \quad \wedge nodes' = Settle([nodes \text{ EXCEPT } \\ & \quad \quad \quad \quad ![n].streams[m][sid] = ClosedStream, \\ & \quad \quad \quad \quad ![m].streams[n][sid] = PutReset(@, s.stop)], \\ & \quad \quad \quad n, m, sid) \end{aligned}$$

— Chain growth —

The peer at the chain tip finds a new block, giving it something to announce. Only the highest peer extends the chain, which keeps the model a single linear chain that the others catch up with.

$$\begin{aligned} MineBlock & \triangleq \\ & \exists n \in InitialPeers : \\ & \quad \wedge Height(n) < MaxBlock \\ & \quad \wedge \forall m \in OtherPeers[n] : Height(m) \leq Height(n) \\ & \quad \wedge nodes' = [nodes \text{ EXCEPT } ![n].blocks = @ \cup \{Height(n) + 1\}] \end{aligned}$$

$$\begin{aligned} Next & \triangleq \\ & \vee Connect \\ & \vee OpenHandshakeStream \\ & \vee SendInitResponder \\ & \vee RecvTypeByte \\ & \vee RecvInit \\ & \vee OpenAnnouncementStream \\ & \vee SendAnnouncement \\ & \vee RecvAnnouncement \\ & \vee FinishAnnouncementStream \end{aligned}$$

$\vee \text{ResetAnnouncementStream}$
 $\vee \text{SendGetHeaders}$
 $\vee \text{ServeGetHeaders}$
 $\vee \text{RecvHeaders}$
 $\vee \text{SendGetBlocks}$
 $\vee \text{ServeGetBlocks}$
 $\vee \text{RecvBlocks}$
 $\vee \text{RecvFin}$
 $\vee \text{RecvReset}$
 $\vee \text{RecvStop}$
 $\vee \text{MineBlock}$

Weak fairness on *Next*: as long as any action is enabled, one is taken.
Mining and restarts are bounded, so every behaviour quiesces and every enabled receive eventually happens. The one loop that is not bounded is a responder refusing pre-handshake announcement streams while the sender keeps reopening them; fairness on *RecvInit* guarantees the responder eventually processes the *init* record that ends that loop.

$$\begin{aligned}
\text{Spec} &\triangleq \\
&\text{Init} \\
&\wedge \Box [\text{Next}]_{\text{vars}} \\
&\wedge \text{WF}_{\text{vars}}(\text{Next}) \\
&\wedge \text{WF}_{\text{vars}}(\text{RecvInit})
\end{aligned}$$

— Liveness —

Every connection eventually completes its handshake (or has been closed).

$$\begin{aligned}
\text{HandshakeCompletes} &\triangleq \\
&\Diamond \Box \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\
&\quad \text{nodes}[n].\text{conn}[m] = \text{"closed"} \vee \text{HandshakeComplete}(n, m)
\end{aligned}$$

Every peer eventually learns every connected peer's final tip.

$$\begin{aligned}
\text{AnnouncementsFlow} &\triangleq \\
&\Diamond \Box \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\
&\quad \text{nodes}[n].\text{conn}[m] = \text{"closed"} \vee \text{nodes}[n].\text{peer_tip}[m] = \text{Height}(m)
\end{aligned}$$

Eventually all peers hold the same chain.

$$\begin{aligned}
\text{EventualConsensus} &\triangleq \\
&\Diamond \Box \forall i, j \in \text{InitialPeers} : \text{nodes}[i].\text{blocks} = \text{nodes}[j].\text{blocks}
\end{aligned}$$

— Safety invariants —

$$\begin{aligned}
\text{TypeOK} &\triangleq \\
&\forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] :
\end{aligned}$$

$$\begin{aligned}
& \wedge \text{nodes}[n].\text{conn}[m] \in \{\text{"none"}, \text{"initiator"}, \text{"responder"}, \text{"closed"}\} \\
& \wedge \text{nodes}[n].\text{close}[m] \in \text{ErrorCodes} \cup \{\text{NoCode}\} \\
& \wedge \text{nodes}[n].\text{version}[m] \in \text{Versions} \cup \{0\} \\
& \wedge \text{nodes}[n].\text{restarts}[m] \in 0 \dots \text{MaxRestarts} \\
& \wedge \text{nodes}[n].\text{score}[m] \in \text{Nat} \\
& \wedge \forall \text{sid} \in \text{StreamIds}(n, m) : \\
& \quad \wedge S(n, m, \text{sid}).\text{status} \in \{\text{"none"}, \text{"open"}, \text{"closed"}\} \\
& \quad \wedge S(n, m, \text{sid}).\text{rtype} \in \text{StreamTypes} \cup \{\text{"unknown"}\} \\
& \quad \wedge S(n, m, \text{sid}).\text{out} \in \{\text{"open"}, \text{"finished"}, \text{"reset"}, \text{"na"}\}
\end{aligned}$$

A stream slot is free on one side exactly when it is free on the other.

$\text{SlotsConsistent} \triangleq$

$$\begin{aligned}
& \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \forall \text{sid} \in \text{StreamIds}(n, m) : \\
& \quad (S(n, m, \text{sid}).\text{status} = \text{"none"}) = (S(m, n, \text{sid}).\text{status} = \text{"none"})
\end{aligned}$$

At most one handshake stream per connection, opened by the initiator

(draft: "Connection Handshake").

$\text{OneHandshakeStream} \triangleq$

$$\begin{aligned}
& \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\
& \quad \wedge \text{Cardinality}(\text{HandshakeStreams}(n, m)) \leq 1 \\
& \quad \wedge \forall \text{sid} \in \text{HandshakeStreams}(n, m) : \\
& \quad \quad \text{IF } \text{sid}[1] = n \text{ THEN } \text{nodes}[n].\text{conn}[m] = \text{"initiator"} \\
& \quad \quad \text{ELSE } \text{nodes}[m].\text{conn}[n] = \text{"initiator"}
\end{aligned}$$

A peer opens no stream before sending its *init* record, and no announcement stream before its handshake completes (draft: "Connection Handshake").

$\text{HandshakeBeforeStreams} \triangleq$

$$\begin{aligned}
& \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\
& \quad \wedge \neg \text{nodes}[n].\text{init_sent}[m] \Rightarrow \\
& \quad \quad \forall k \in 1 \dots \text{MaxStreams} : S(n, m, \langle n, k \rangle).\text{status} = \text{"none"} \\
& \quad \wedge \neg \text{HandshakeComplete}(n, m) \Rightarrow \\
& \quad \quad \forall k \in 1 \dots \text{MaxStreams} : S(n, m, \langle n, k \rangle).\text{rtype} \notin \text{AnnTypes}
\end{aligned}$$

Once *n* has received *m*'s *init*, the negotiated version is the minimum of the two advertised versions (draft: "Protocol Versioning").

$\text{NegotiatedVersionIsMin} \triangleq$

$$\begin{aligned}
& \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\
& \quad \text{HandshakeComplete}(n, m) \Rightarrow \\
& \quad \quad \text{nodes}[n].\text{version}[m] = \text{Min}(\text{nodes}[n].\text{my_version}, \text{nodes}[m].\text{my_version})
\end{aligned}$$

A sender never has two announcement streams of one type open toward a peer (draft: "Announcement Streams").

$\text{SenderSingleton} \triangleq$

$$\begin{aligned}
& \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \forall t \in \text{AnnTypes} : \\
& \quad \text{Cardinality}(\text{LiveAnn}(n, m, t)) \leq 1
\end{aligned}$$

OBSOLETE is only ever used against a peer that advertised an old version.

$CloseJustified \triangleq$
 $\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $nodes[n].close[m] = \text{"OBSOLETE"} \Rightarrow$
 $\quad \vee nodes[n].my_version < MinVersion$
 $\quad \vee nodes[m].my_version < MinVersion$

Blocks are always a contiguous chain from genesis.

$BlocksContiguous \triangleq$
 $\forall n \in InitialPeers : nodes[n].blocks = 1 \dots Height(n)$

Request streams are only opened after the handshake completes, one at a time per peer (draft: "Connection Handshake"; ZIP – 204 in-transit limit).

$RequestsAfterHandshake \triangleq$
 $\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $\quad \wedge OpenReq(n, m) \neq \{\} \Rightarrow HandshakeComplete(n, m)$
 $\quad \wedge Cardinality(OpenReq(n, m)) \leq 1$

Responses in flight respect the draft's bounds: at most $MaxHeaders$ contiguous headers, at most $MaxBlocksPerRequest$ hashes per get-blocks.

$ResponsesBounded \triangleq$
 $\forall n \in InitialPeers : \forall m \in OtherPeers[n] : \forall sid \in StreamIds(n, m) :$
 $\quad \forall i \in 1 \dots Len(S(n, m, sid).inq) :$
 $\quad \text{LET } x \triangleq S(n, m, sid).inq[i]$
 $\quad \text{IN } \quad \wedge x.kind = \text{"headers"} \Rightarrow Len(x.heights) \leq MaxHeaders \wedge Contiguous(x.heights)$
 $\quad \quad \wedge x.kind = \text{"get-blocks"} \Rightarrow Len(x.heights) \leq MaxBlocksPerRequest$
 $\quad \quad \wedge x.kind = \text{"blocks"} \Rightarrow Len(x.heights) \leq MaxBlocksPerRequest$

Honest peers never earn a misbehavior penalty.

$NoHonestPenalty \triangleq$
 $\forall n \in InitialPeers : \forall m \in OtherPeers[n] : nodes[n].score[m] = 0$

No interleaving of conformant behaviour ends in a protocol error. There are no adversarial actions in this phase, so any close other than *OBSOLETE* is a disagreement between honest peers about the draft's rules.

$NoHonestProtocolError \triangleq$
 $\forall n \in InitialPeers : \forall m \in OtherPeers[n] :$
 $\quad nodes[n].close[m] \in \{NoCode, \text{"OBSOLETE"}\}$